

PRANVEER SINGH INSTITUTE OF TECHNOLOGY

Major Project Proposal

Team Id: 27_CSAI_4A_09

Team Details:

S.No.	Full Name	Roll No	Branch & Section	Mob No
1	Bhavya Gupta	2301641520064	CS AI 3-A	9569461013
2	Bhumika Mishra	2301641520065	CS AI 3-A	9336979743
3	Divyansh Gaur	2301641520070	CS AI 3-A	9026560306
4	Divyanshi Singh	2301641520072	CS AI 3-A	9548482611

Project Title:

AURIX: An Agentic Unified Risk Intelligence Platform with Autonomous Detection and Remediation

Domain:

1. Software-Web Application
2. Artificial Intelligence/Machine Learning/Deep Learning
3. Gen AI/Agentic AI

Problem Statement:

Modern software development, accelerated by AI-driven tools like Claude Code and Cursor, allows teams to ship code at an unprecedented pace. However, this speed creates a massive "**Security Gap**": while code is deployed daily, deep security validation and penetration testing typically happen only once a year. For the other 364 days, organizations unknowingly ship vulnerabilities to production.

Furthermore, existing security workflows are currently hindered by three core challenges:

- **Tool Fragmentation:** Scanners for code (SAST), dependencies (SCA), and infrastructure (Docker) generate siloed results that are difficult to analyze, correlate, and prioritize.
- **Alert Fatigue:** Traditional tools produce a high volume of theoretical risks but fail to "prove" if a vulnerability is actually exploitable in a live environment.

PRANVEER SINGH INSTITUTE OF TECHNOLOGY

- **The Expertise Gap:** Many developers lack the specialized security knowledge required to interpret complex reports and manually verify if a patch truly neutralizes a threat.

There is a critical need for an autonomous, unified platform that bridges this gap by providing on-demand, agentic pentesting and self-healing remediation for every build ensuring that security defense moves at the same speed as software innovation.

Proposed Solution:

We propose AURIX: An Agentic Unified Risk Intelligence Platform with Autonomous Detection and Remediation. Designed to bridge the "Security Gap," AURIX functions as a self-healing ecosystem that provides autonomous validation and remediation for every code build.

The solution is built on three core pillars:

- **Unified Multi-Layer Engine:** A secure, Docker-isolated engine that executes SAST, SCA, Secret Detection, and Infrastructure audits (Opengrep, Trivy, Gitleaks, Hadolint etc.) in parallel. It normalizes fragmented tool outputs into a single intelligence schema to identify cross-layer risks that traditional tools miss.
- **Agentic Red-vs-Blue Wargaming:** A dual-agent simulation designed to eliminate false positives. An **Autonomous Red Agent** reads the source code to identify attack paths and generates executable **Proof-of-Concept (PoC)** scripts to prove exploitability. Simultaneously, a **Blue Agent** creates a remediation patch, which is then verified against the exploit in a hardened sandbox to ensure 100% effectiveness.
- **Visual Intelligence & Self-Healing:** The system translates complex technical data into a **File-Level Risk Heat Map** for instant prioritization of "Hot Zones." Once a fix is verified in the wargaming loop, the platform completes the lifecycle via **Autonomous PR Remediation**, automatically opening a Pull Request on GitHub to secure the project without manual intervention.

By integrating these features, **AURIX** ensures that security defense moves at the same speed as AI-accelerated development, providing a "verified secure" codebase for every release.

Unique/Distinctive feature of the solution:

1.Explain the uniqueness and distinctive features of the solution

AURIX is not just a scanner that finds problems; it is an intelligent system that **proves** and **fixes** them. Here is what makes it unique:

- **The "All-in-One" Intelligence Engine:** Most tools only look at one thing (like just code or just libraries). AURIX scans everything at once code, third-party

PRANVEER SINGH INSTITUTE OF TECHNOLOGY

libraries, hidden secrets, and Docker setup files. It connects the dots to find "hidden" risks that other tools miss.

- **Red-vs-Blue Wargaming:** This is our most unique feature. AURIX has two AI agents:
 - **Red Agent (Attacker):** Tries to hack your code to prove a bug is real.
 - **Blue Agent (Defender):** Creates a patch to fix that specific bug. They "battle" in a secure environment to make sure the fix actually stops the attack.
- **Real Proof (Not Just Alerts):** Instead of giving you a long list of "maybe" bugs, AURIX gives you an executable script (a Proof-of-Concept). It shows you exactly how a hacker would break in, so you don't waste time on false alarms.
- **Guaranteed Fixes in a Sandbox:** We use a high-security "sandbox" (gVisor) to test our repairs. A fix is only shown to you if it successfully stops the Red Agent's attack. This ensures that the patch you get actually works and doesn't break your app.
- **AI "Jury" System:** To stop "alert fatigue," we use multiple AI agents to vote on every bug found. Only high-priority, confirmed issues make it to your dashboard.
- **Visual Risk Heat Map:** AURIX changes boring tables into a "Heat Map." You can see exactly which files are "hot" (most dangerous), making it easy to know where to start fixing first.
- **Hands-Free "Self-Healing":** Once a fix is proven to work in our wargame, AURIX automatically opens a "Pull Request" on GitHub for you. You don't have to write a single line of security code, you just review and merge.

2.How your proposed solution is different from similar kind of products by the competitors

The following table compares **AURIX** with traditional industry leaders (such as Snyk, Wiz, and Checkmarx) to highlight its competitive advantages in the modern security landscape.

Feature	Traditional Competitors (Snyk / Wiz / Checkmarx)	AURIX Platform	The AURIX Advantage
Tool Integration	Often requires multiple separate tools for code, libraries, and secrets.	Unified Multi-Layer Engine: Scans code, dependencies, and infrastructure in one parallel process.	Contextual Awareness: Identifies risks that span across different layers which siloed tools miss.

PRANVEER SINGH INSTITUTE OF TECHNOLOGY

Accuracy & Validation	Uses static, rule-based logic that often leads to many "False Positives."	Agentic Red-vs-Blue Wargaming: AI agents simulate real attacks to prove if a bug is actually exploitable.	Zero Noise: Eliminates alert fatigue by only prioritizing confirmed, reachable threats.
Proof of Risk	Provides text-based descriptions and theoretical risk scores.	Autonomous PoC Scripts: Generates an executable script that demonstrates the exploit in action.	Undeniable Evidence: Developers see exactly how a bug works before they are asked to fix it.
Remediation & Testing	Offers "suggested" code fixes that may break the application or fail to work.	Closed-Loop Sandbox Verification: Tests every patch against the exploit in a secure gVisor environment.	Guaranteed Fixes: Ensures the patch actually closes the security gap without breaking the build.
Visualization	Relies on standard dashboard lists and tables.	Visual Risk Heat-Mapping: Projects risk "temperature" directly onto the file structure.	Fast Prioritization: Instant visual identification of "Hot Zones" for immediate attention.
Workflow Strategy	Reactive detection; notifies the user of a problem after it is found.	Autonomous Self-Healing: Automatically finds, proves, fixes, and opens a Pull Request on GitHub.	End-to-End Automation: Moves from detection to resolution without any manual developer intervention.

PRANVEER SINGH INSTITUTE OF TECHNOLOGY

Tools/Technology Uses:

1) Hardware Requirements

- a) Processor: Intel Core i5/i7 10th Gen or higher / AMD Ryzen 5/7 5000 series
- b) RAM: Minimum 16 GB
- c) Storage: Minimum 256 GB SSD
- d) Graphics Card: Not required (Integrated graphics sufficient)
- e) Others: Stable internet connection for repository access and dependency scanning

2) Software Requirements

- a) Operating System: Linux (Ubuntu 22.04 LTS) or Windows with WSL2.
- b) Development Tools: Visual Studio Code, Python 3.11+, Node.js, Docker, gVisor, Git
- c) Database: PostgreSQL, Redis
- d) Web Browser: Google Chrome / Microsoft Edge / Firefox
- e) Others: Semgrep (SAST scanning), Trivy (SCA Scanning), Gitleaks(secret detection), Hadolint(Dockerfile linting), GitHub API for repository access

3) Others

React.js / Next.js for frontend dashboard, Tailwind CSS or shadcn UI for interface design, Recharts or Chart libraries for visualization, Google Gemini Pro or GPT-4o API integration for vulnerability explanation and remediation suggestions.

PRANVEER SINGH INSTITUTE OF TECHNOLOGY

Proposal Evaluation:

1. Right Identification of the Problem (Appropriate selection of the problem)?
a) Excellent b) Good c) Needs Improvement d) Unacceptable

2. Relevance of the Solution (Adequately addressing the problem/need)?
a) Excellent b) Good c) Needs Improvement d) Unacceptable

3. Innovativeness in the Solution (Distinctive innovative components/features of the solution)?
a) Excellent b) Good c) Needs Improvement d) Unacceptable

4. Uniqueness of the Solution (Intellectual Property Component)?
a) Excellent b) Good c) Needs Improvement d) Unacceptable

Improvements/ Suggestions by the Evaluator:

1.	
2.	
3.	
4.	

Name of Faculty In-charge: Dr. Jyoti Dua

Designation: Associate Professor

Signature with Date: